RIPHAH INTERNATIONAL UNIVERSITY

DATA STRUCTURES LAB

Lab Task 13

Name: Aroosa Feroz

SAP ID: 68997

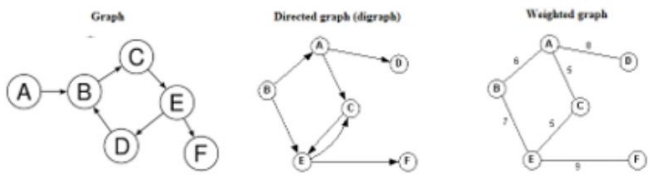
Submitted to: Ma'am Nehan Sahar

Subject: Data Structures Lab

DEPARTMENT OF COMPUTER SCIENCE

RIPHAH INTERNATIONAL UNIVERSITY, GULDBERG GREENS CAMPUS ISLAMABAD

Lab Task 1:
• Make Adjacency Matrix of above graphs.
• Make Adjacency list of above graphs.
• Perform BFS.
• Perform DFS.



Ans:

<u>Graph 1</u>

Given graph is an undirected graph.

Edges are:

A-B
B-C
B-D
C-E
D-E
E-F

1. Adjacency Matrix

	A	B	C	D	E	F
A	0	1	0	0	0	0
B	1	0	1	1	0	0
C	0	1	0	0	1	0
D	0	1	0	0	1	0
E	0	0	1	1	0	1
F	0	0	0	0	1	0

2. Adjacency List

A -> B
B -> A, C, D
C -> B, E
D -> B, E
E -> C, D, F
F -> E

3. BFS (starting from A)

Steps:

- Start: A
- Visit neighbors -> B
- From B -> C, D
- From C -> E
- From D -> (E already visited)
- From E -> F

Traversal order:

A -> B -> C -> E -> F -> E -> C -> B -> D -> E -> D -> B -> A

4. DFS (starting from A)

Step-by-step movement:

A -> B -> C -> E -> F
back to E
back to C
back to B
-> D
back to B
back to A

Traversal order:

A -> B -> A -> C -> B -> D -> B -> E -> C -> F

Graph 2

Given graph 2 is a directed graph

Edges are

B -> A
B -> E
A -> D
A -> C
C -> E
E -> F

1.Adjacency Matrix

	A	B	C	D	E	F
A	0	0	1	1	0	0
B	1	0	0	0	1	0
C	0	0	0	0	1	0
D	0	0	0	0	0	0
E	0	0	0	0	0	1
F	0	0	0	0	0	0

2. Adjacency List

A -> C, D
B -> A, E
C -> E
D -> no connection
E -> F
F -> no connection

3. BFS (start from B)
Steps:

- B -> A

- B -> E
- A -> C
- A -> D
- E -> F

Path:
B -> A -> B -> E -> B -> C -> A -> D -> A -> F

4. DFS (start from B)
Steps:

- B -> A
- A -> C
- C -> E
- E -> F
- Backtrack: F -> E -> C -> A
- A -> D
- Backtrack: D -> A -> B

Traversal Order:
B -> A -> C -> E -> F -> E -> C -> A -> D -> A -> B

Graph 3

Given graph 3 is a weighted graph

Edges are

A–B
A–C
A–D
B–E
C–E
E–F

1. Adjacency Matrix

	A	**B**	**C**	**D**	**E**	**F**
A	0	1	1	1	0	0
B	1	0	0	0	1	0
C	1	0	0	0	1	0
D	1	0	0	0	0	0
E	0	1	1	0	0	1
F	0	0	0	0	1	0

2. Adjacency List

A -> B, C, D
B -> A, E
C -> A, E
D -> A
E -> B, C, F
F -> E

3. BFS (start from A)

Steps:

- A -> B
- A -> C
- A -> D
- B -> E
- E -> F

Traversal Order:
A -> B -> A -> C -> A -> D -> A -> E -> B -> F

4. DFS (start from A)

Steps:

- A -> B
- B -> E
- E -> C
- Backtrack: C -> E
- E -> F
- Backtrack: F -> E -> B -> A
- A -> D
- Backtrack: D -> A

Traversal Order:

A -> B -> E -> C -> E -> F -> E -> B -> A -> D -> A

Task 2:

Draw a graph modes, starting the type of graph used, to represent airline routs where every day there are four flights from Boston to Newark, two flights from Newark to Boston, three flights from Newark to Miami, two flights from Miami to Newark, one flight from Newark to Detroit, two flights from Detroit to Newark, three flights from Newark to Washington, two flights from Washington to Newark, and one flight from Washington to Miami, with
(1) an edge between vertices representing cities that have a flight between them (in either direction)
(2) an edge between vertices representing cities for each flight that operates between them) in either direction)
(3) an edge between vertices representing cities for each flight that operates between them (in either direction) plus a loop for a special sightseeing trip that takes off and lands in Miami
Ans:

Cities:

Boston, Newark, Miami, Detroit, Washington

Flights:

- Boston -> Newark = 4
- Newark -> Boston = 2
- Newark -> Miami = 3
- Miami -> Newark = 2
- Newark -> Detroit = 1
- Detroit -> Newark = 2
- Newark -> Washington = 3
- Washington -> Newark = 2

- Washington -> Miami = 1

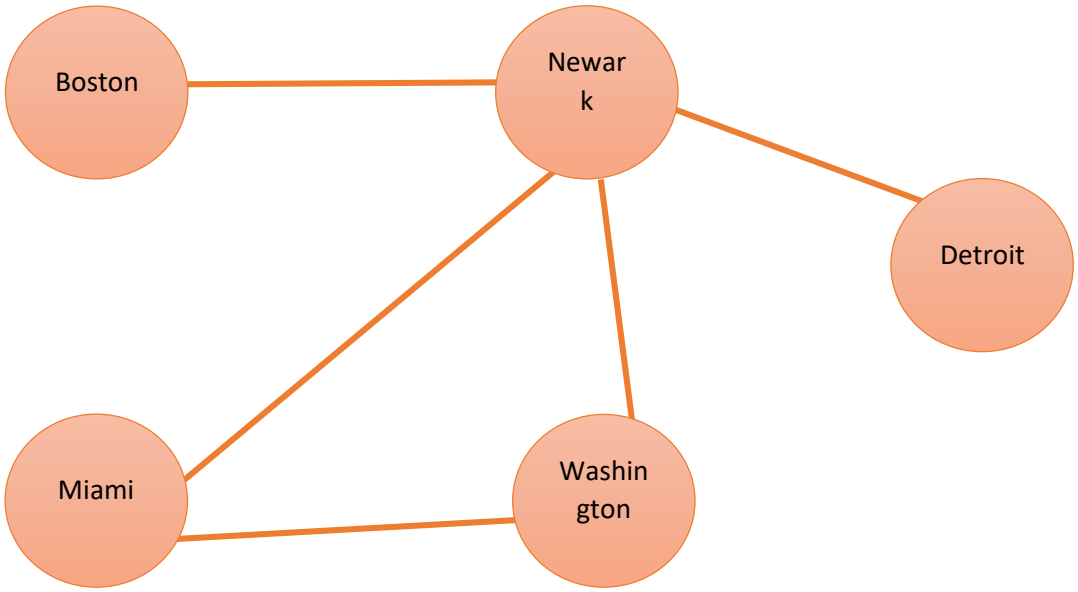
(1) Simple Graph (Unweighted, Undirected)

Given that only check if cities have flight between them and in either direction so it is an undirected graph.
Ignore direction + number of flights only show the connection between cities i.e flights by one line.

Edges:

- Boston - Newark
- Newark - Miami
- Newark - Detroit
- Newark -Washington
- Washington -Miami

<u>Graph:</u>



(2) Weighted Graph (Weighted, Undirected)

Given that only check if cities have flight between them and in either direction so it is an undirected graph also an edge between cities for each flight that operates between them.

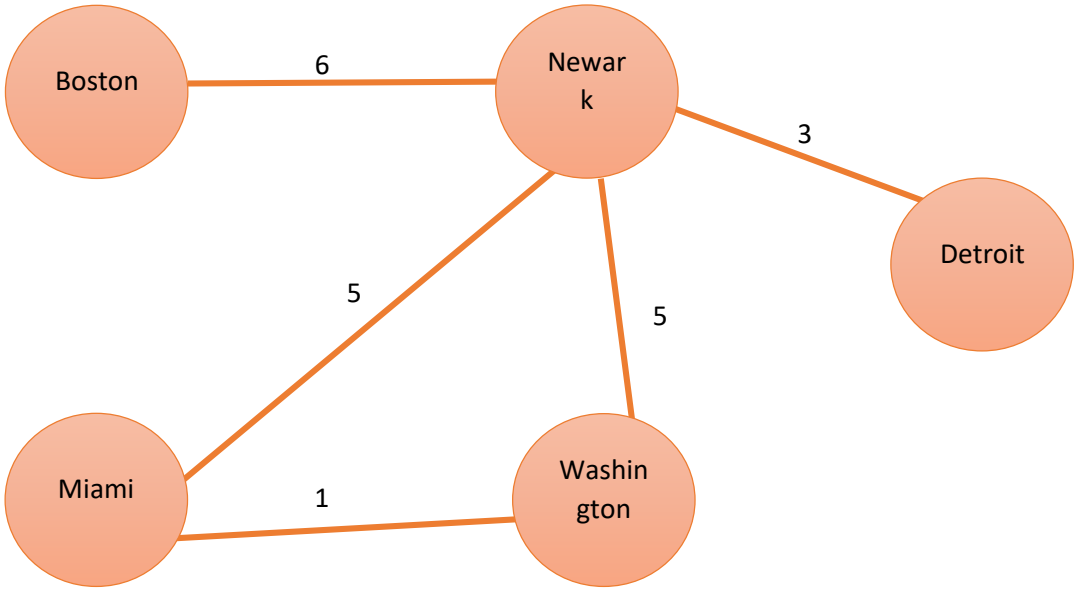
Ignore direction and number on top of edges between cities is equal to number of flights.

Edges:

- Boston - Newark 4+2 =6
- Newark - Miami 3+2=5
- Newark - Detroit 1+2=3
- Newark -Washington 3+2=5
- Washington –Miami =1

For edges we consider both paths from from city say A to other city say B and also from B to A.

<u>Graph:</u>



(3) Weighted Graph with loop (Weighted, Undirected)

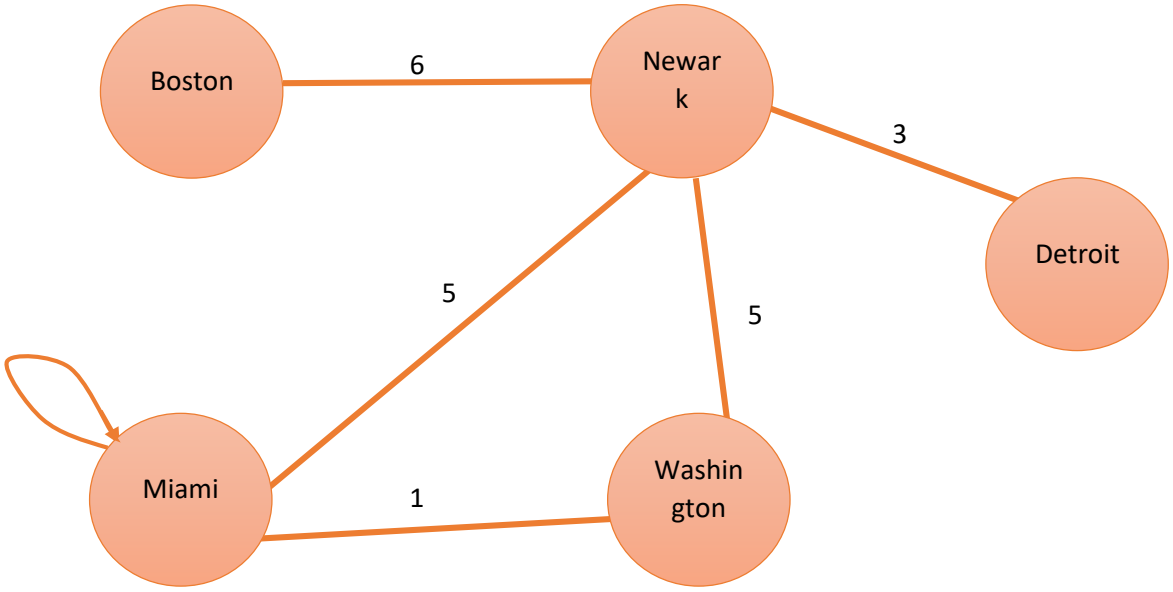
Given that only check if cities have flight between them and in either direction so it is an undirected graph also an edge between cities for each flight that operates between them.
Ignore direction and number on top of edges between cities is equal to number of flights. Also the loop is for Miami flight that takes off and lands in miami

Edges:

- Boston - Newark 4+2 =6
- Newark - Miami 3+2=5
- Newark - Detroit 1+2=3
- Newark -Washington 3+2=5
- Washington –Miami =1
- Miami-Miami=1 (loop)

For edges we consider both paths from from city say A to other city say B and also from B to A.

<u>Graph:</u>



Task 3:

Describe a graph model that represents whether each person at a class knows the name of each other person at the class. Should the edges be directed or undirected? Should multiple edges be allowed? Should loops be allowed?

Ans:

Graph Model:

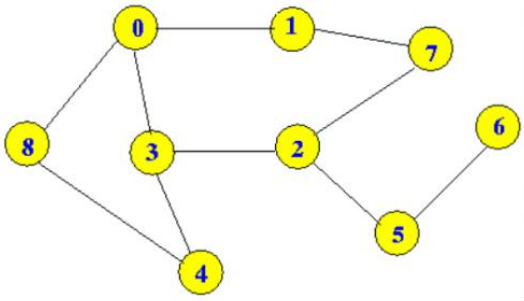
A graph can be used to represent whether each person in a class knows the name of another person.

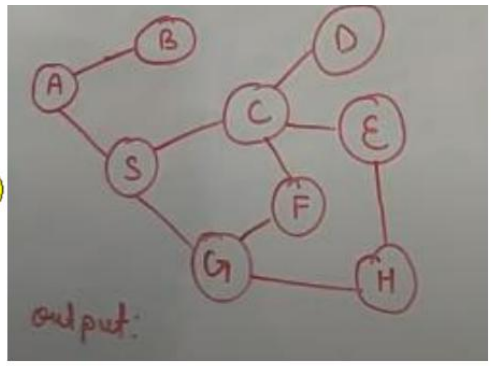
- **Vertices:** Each vertex represents a person in the class.
- **Edges:** A directed edge from one vertex to another (A -> B) represents that person A knows the name of person B.

This graph should be a directed graph (digraph) because knowing someone's name is not always mutual.

- **Multiple edges:** Not allowed, because between any two people there is only one possible relationship knows or does not know.
- **Loops:** Not allowed, because a person knowing their own name is not considered in this model.

Task 4:





• Perform BFS.

• Perform DFS

Ans:

Graph 1 (0–8)

Start: **0**

DFS (Steps + Full Path with backtracking)

Steps:

1. 0 -> 1
2. 1 -> 7
3. 7 -> 2

4. 2 -> 5
5. 5 -> 6
6. Backtrack to 5 -> 2
7. 2 -> 3
8. 3 -> 4
9. Backtrack to 3 -> 2 -> 7 -> 1 -> 0
10. 0 -> 8
11. Backtrack to 0

Full Path:
0 -> 1 -> 7 -> 2 -> 5 -> 6 -> 5 -> 2 -> 3 -> 4 -> 3 -> 2 -> 7 -> 1 -> 0 -> 8 -> 0

BFS (Movement-style Path)

Steps:

1. 0 -> (1, 3, 8)
2. 1 -> 7
3. 3 -> 2, 4
4. 8 -> —
5. 7 -> 6
6. 2 -> 5

Full Path:
0 -> 1 -> 0 -> 3 -> 0 -> 8 -> 0 -> 7 -> 1 -> 2 -> 3 -> 4 -> 3 -> 6 -> 7 -> 5 -> 2

Graph 2 (A–H)

Start: **A**

DFS (Steps + Full Path with backtracking)

Steps:

1. A -> B
2. Backtrack to A
3. A -> S
4. S -> C
5. C -> D
6. Backtrack to C
7. C -> E
8. E -> H
9. H -> G
10. G -> F
11. Backtrack all the way to A

Full Path:
A -> B -> A -> S -> C -> D -> C -> E -> H -> G -> F -> G -> H -> E -> C -> S -> A

BFS (Movement-style Path)

Steps:

1. A -> (B, S)
2. B -> —
3. S -> (C, G)
4. C -> (D, E, F)
5. G -> H

Full Path:
A -> B -> A -> S -> A -> C -> S -> G -> S -> D -> C -> E -> C -> F -> C -> H -> G

Thank You